CMSC131: Strings, Expressions, and Input

Chapter 2

By Naren Ayinala

May 2026

Contents

1 Assignment and Equality Are Different Operations **2**

1.1 Assignment: `=` . . . 2

1.2 Comparison: `==` . . . 2

1.3 Comparing Primitive Types . . . 2

2 Compound Assignment Operators **3**

3 What Is a String? **4**

3.1 Why `==` Is Different for Strings . . . 4

3.2 The String Pool . . . 5

4 String Concatenation and `+=` **5**

4.1 Why `-=`, `*=`, and `/=` Do Not Work on Strings . . . 5

4.2 Order Matters When Combining Numbers and Strings . . . 6

5 How Strings Work in Memory **6**

5.1 String Methods Return Results . . . 7

6 StringBuilder: A Mutable Sequence of Characters **7**

6.1 Common StringBuilder Methods . . . 8

7 StringBuffer: Similar to StringBuilder, with Synchronization **8**

8 Reading Input with Scanner **9**

9 Worked Example: Build a Sentence **9**

10 Summary: Choosing the Correct Operation **10**

1 Assignment and Equality Are Different Operations

Before we work with Strings, we need to understand three symbols that look similar but do completely different jobs: `=`, `==`, and `equals()`.

1.1 Assignment: =

A single equals sign means **assignment**. It evaluates the expression on the right and stores the result in the variable on the left. It does not ask whether the two sides are equal.

```
int age = 18; // Store 18 in age.
age = 19; // Replace the old value with 19.
int copy = age; // Copy the current value (19) into copy.
age = 20;
System.out.println(copy); // 19, not 20
```

The expression on the right is evaluated *before* the assignment takes place. For example, `x = x + 1` is valid Java: Java reads the old value of `x`, adds one, and stores the result back into `x`. This is not a mathematical equation asserting that $x = x + 1$.

1.2 Comparison: ==

Two equals signs mean **comparison**. The expression `a == b` produces a `boolean`: either `true` or `false`. For primitive numeric values, Java compares their values (after any required numeric promotion).

```
int a = 7;
int b = 7;
int c = 10;
System.out.println(a == b); // true
System.out.println(a == c); // false
System.out.println(a != c); // true
boolean same = (a == b); // Store the comparison result.
```

The operator `!=` means "not equal." Comparisons are often used in `if` statements and loops.

```
int temperature = 30;
if (temperature == 30) {
    System.out.println("Exactly 30");
}
```

1.3 Comparing Primitive Types

A **primitive variable** directly stores a primitive value. For `int`, `char`, and `boolean`, `==` tests whether those values match.

```
int x = 12;
int y = 12;
char first = 'A';
char second = 'B';
boolean ready = true;
System.out.println(x == y); // true
System.out.println(first == second); // false
```

```
System.out.println(ready == true); // true
System.out.println(12 == 12.0); // true: numeric promotion
```

Floating-point values require care: many decimal fractions cannot be represented exactly in binary floating point.

```
double result = 0.1 + 0.2;
System.out.println(result == 0.3); // false
System.out.println(Math.abs(result - 0.3) < 1e-9); // true
```

The last line tests whether the values are *close enough* for a chosen tolerance. It is not a universal comparison rule; the appropriate tolerance depends on the application.

Syntax	Meaning	Result
`x = 5`	Assign the value 5 to `x`	Assignment expression
`x == 5`	Ask whether `x` equals 5	`boolean`
`x != 5`	Ask whether `x` differs from 5	`boolean`
`x += 5`	Add 5 to `x` and assign back	Assignment expression

2 Compound Assignment Operators

When we repeatedly update a variable, Java provides shorter forms of common assignments. For ordinary `int` examples, `x += 3` behaves like `x = x + 3`.

Operator	Long form	Meaning
`x += 4`	`x = x + 4`	Addition
`x -= 4`	`x = x - 4`	Subtraction
`x *= 4`	`x = x * 4`	Multiplication
`x /= 4`	`x = x / 4`	Division
`x %= 4`	`x = x % 4`	Remainder

```
int number = 20;
number += 5; // 25
number -= 3; // 22
number *= 2; // 44
number /= 4; // 11 (integer division)
number %= 6; // 5
System.out.println(number); // 5
```

These operators evaluate the left-hand variable once and perform an assignment conversion. Therefore, they are *not always* identical to spelling out the long form for every type:

```
byte small = 10;
small += 2; // Legal: implicit narrowing in compound assignment.
// small = small + 2; // Compile error: expression has type int.
```

For now, think of compound assignment as a convenient way to update the existing variable, while remembering that Java's type rules still apply.

3 What Is a String?

A `String` is an object representing a sequence of characters. Unlike `char`, which represents one UTF-16 code unit, a String can contain zero, one, or many characters. A String literal is enclosed in double quotes, while a `char` literal uses single quotes.

```
char letter = 'J';
String word = "Java";
String empty = "";
String sentence = "I am learning Java.";
System.out.println(word.length()); // 4
System.out.println(word.charAt(0)); // J
```

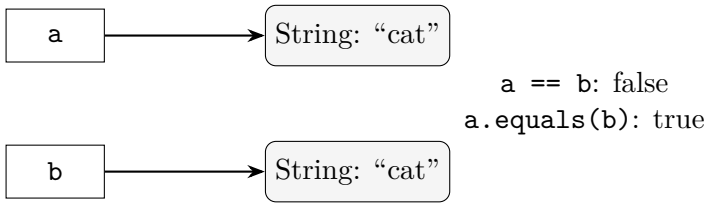
Strings are **reference types**, not primitive types. A String variable holds a reference to a String object. The String object holds the text.

3.1 Why == Is Different for Strings

For primitive types, `==` compares primitive values. For reference types, `==` compares **identity**: do both references refer to the exact same object? Two separate String objects can contain identical characters while being different objects.

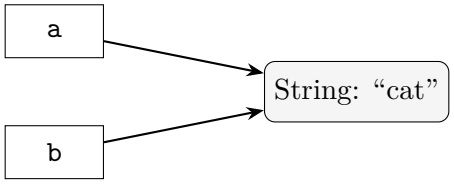
```
String a = new String("cat");
String b = new String("cat");
System.out.println(a == b); // false: different objects
System.out.println(a.equals(b)); // true: same characters
```

The `String` class defines `equals()` to compare the *contents* of the strings. Thus, if your question is "do these two Strings contain the same text?", use `equals()`.



Now compare that with two references pointing to *one* object:

```
String a = new String("cat");
String b = a;
System.out.println(a == b); // true
System.out.println(a.equals(b)); // true
```



3.2 The String Pool

Java maintains a pool of canonical String objects for literals and interned Strings. This allows equal literals to share one object:

```
String first = "hello";
String second = "hello";
System.out.println(first == second); // true: pooled literals
```

However, **never use this as a reason to compare text with ==**. A String might come from a constructor, input, a method, or another computation. Those Strings can contain the same text without sharing an object.

```
String first = "hello";
String second = new String("hello");
System.out.println(first == second); // false
System.out.println(first.equals(second)); // true
```

If a String reference might be `null`, calling a method on it causes a `NullPointerException`. A constant-first comparison can avoid that:

```
String input = null;
System.out.println("yes".equals(input)); // false
// input.equals("yes"); // NullPointerException
```

4 String Concatenation and +=

The `+` operator has a special meaning when at least one operand is a String: it **concatenates** text, joining the pieces together.

```
String first = "Hello";
String message = first + " world";
System.out.println(message); // Hello world
```

We can also use `+=` to append to the value stored in a String variable:

```
String text = "CMSC";
text += "131";
text += " Java";
System.out.println(text); // CMSC131 Java
```

The expression `text += "131"` is similar in effect to `text = text + "131"`. But Strings are immutable: Java constructs a String with the combined text and updates the variable to refer to the result. It does *not* insert characters into the old String object.

4.1 Why -=, *=, and /= Do Not Work on Strings

Java defines String concatenation using `+`, and therefore String `+=` is valid. It does *not* define subtraction, multiplication, or division for Strings.

```
String name = "Java";
name += " Programming"; // Legal
// name -= "Java"; // Compile error
// name *= 2; // Compile error
// name /= 2; // Compile error
```

If you want to remove text, use String methods such as `replace()`, `substring()`, or a mutable character-sequence class. For example:

```
String text = "Java Programming";
text = text.replace("Java ", "");
System.out.println(text); // Programming
```

This returns a new String; it is not a String subtraction operator.

4.2 Order Matters When Combining Numbers and Strings

Java evaluates + expressions from left to right. Until a String is encountered, numeric addition can happen; after a String is involved, concatenation takes over.

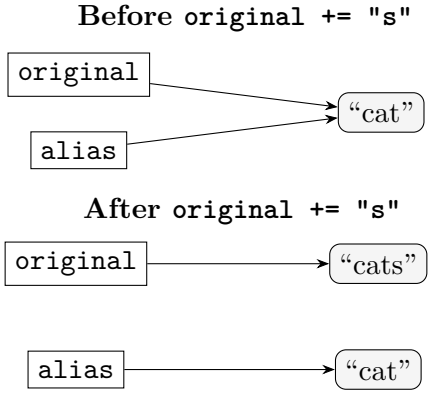
```
System.out.println(2 + 3 + " cats"); // 5 cats
System.out.println("Cats: " + 2 + 3); // Cats: 23
System.out.println("Cats: " + (2 + 3)); // Cats: 5
```

Parentheses make the intended calculation explicit.

5 How Strings Work in Memory

String objects are **immutable**. Once a particular String object has been created, its character sequence cannot be changed. A variable can be *reassigned* to another String object, but that does not mutate the original object.

```
String original = "cat";
String alias = original;
original += "s";
System.out.println(original); // cats
System.out.println(alias); // cat
```



Notice that `alias` still points to the original text. Reassigning `original` does not change other references.

5.1 String Methods Return Results

Because Strings are immutable, methods such as `toUpperCase()`, `trim()`, and `replace()` do not alter the existing String.

```
String name = " ada ";
name.trim();
System.out.println(name); // " ada "
name = name.trim();
System.out.println(name); // "ada"
```

Other useful methods include:

Method	What it does
`length()`	Number of UTF-16 code units
`charAt(i)`	Character code unit at zero-based index `i`
`substring(a,b)`	Characters from index `a` (inclusive) to `b` (exclusive)
`indexOf("x")`	Index of first occurrence, or -1 if absent
`equals(other)`	Case-sensitive content equality
`equalsIgnoreCase(other)`	Case-insensitive equality under Java's rules
`compareTo(other)`	Lexicographic comparison
`toUpperCase()`	Uppercase result String
`replace(old,new)`	Replace occurrences and return result

```
String word = "computer";
System.out.println(word.length()); // 8
System.out.println(word.charAt(2)); // m
System.out.println(word.substring(0, 4)); // comp
System.out.println(word.indexOf("put")); // 3
System.out.println(word.compareTo("computer")); // 0
```

6 StringBuilder: A Mutable Sequence of Characters

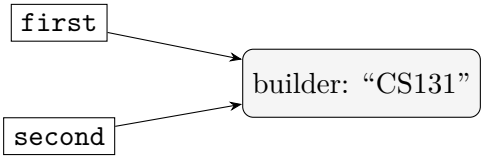
If you need to repeatedly append, insert, delete, or replace characters, creating many intermediate immutable Strings can be inefficient. Java provides `StringBuilder`, which stores a **mutable** sequence of characters.

```
StringBuilder builder = new StringBuilder("Java");
builder.append(" is");
builder.append(" fun");
System.out.println(builder); // Java is fun
builder.insert(4, " programming");
System.out.println(builder); // Java programming is fun
```

Unlike `String`, `StringBuilder` methods such as `append()` modify the same builder object.

```
StringBuilder first = new StringBuilder("CS");
StringBuilder second = first;
```

```
first.append("131");
System.out.println(second); // CS131
```



Both variables refer to the same mutable object. Therefore, appending through one reference is visible through the other.

6.1 Common StringBuilder Methods

Method	Effect
`append(x)`	Add text to the end
`insert(i,x)`	Insert at index `i`
`delete(a,b)`	Remove characters in range `[a,b)`
`deleteCharAt(i)`	Remove one character
`replace(a,b,s)`	Replace range with `s`
`reverse()`	Reverse the sequence of UTF-16 code units
`setCharAt(i,c)`	Replace one character
`toString()`	Produce a String with the current text

```
StringBuilder sb = new StringBuilder("abc");
sb.append("def"); // abcdef
sb.delete(1, 3); // adef
sb.insert(1, "BC"); // aBCdef
sb.setCharAt(0, 'A'); // ABCdef
String answer = sb.toString();
System.out.println(answer); // ABCdef
```

7 StringBuffer: Similar to StringBuilder, with Synchronization

`StringBuffer` is another mutable character-sequence class. It has many of the same methods as `StringBuilder`, including `append()`, `insert()`, `delete()`, and `toString()`.

```
StringBuffer buffer = new StringBuffer("Hello");
buffer.append(" world");
System.out.println(buffer.toString()); // Hello world
```

The important difference is that `StringBuffer`'s individual methods are synchronized. This provides coordination for individual method calls when multiple threads share a buffer. `StringBuilder` is generally preferred when such synchronization is unnecessary, because it avoids that overhead. Synchronization of individual methods does not automatically make an entire multi-step operation atomic.

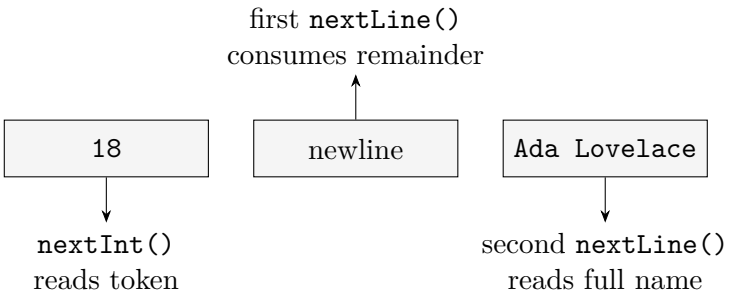
Property	`String`	`StringBuilder`	`StringBuffer`
Mutable?	No	Yes	Yes
`append()` changes object?	No such method	Yes	Yes
Typical use	Fixed text values	Building/editing text	Shared mutable text requiring method synchronization
Thread synchronization	Immutable object	Not synchronized	Synchronized methods

8 Reading Input with Scanner

To read from standard input, we can create a `Scanner` that reads `System.in`. Different methods read different portions of the input.

```
import java.util.Scanner;
public class InputExample {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Age: ");
        int age = scanner.nextInt();
        scanner.nextLine(); // consume rest of line
        System.out.print("Name: ");
        String name = scanner.nextLine();
        System.out.println(name + " is " + age);
    }
}
```

`nextInt()` reads an integer token but does not consume the following line terminator. If we call `nextLine()` immediately after it, that call often consumes only the remainder of the current line. An extra `nextLine()` deliberately moves past that remainder before reading the next full line.



9 Worked Example: Build a Sentence

This example combines primitive variables, compound assignment, Strings, and StringBuilder.

```
int courses = 2;
courses += 1; // courses is now 3
String intro = "I take ";
intro += courses; // "I take 3"
intro += " courses."; // "I take 3 courses."
StringBuilder message = new StringBuilder(intro);
message.append(" One is CMSC131.");
```

```
System.out.println(message.toString());
```

The printed output is:

```
I take 3 courses. One is CMSC131.
```

Here `courses += 1` performs numeric addition, whereas `intro += courses` performs String concatenation because the left-hand variable is a String. Finally, `message.append(...)` mutates the existing StringBuilder.

10 Summary: Choosing the Correct Operation

Goal	Use
Store a new value	`=`
Compare primitive values	`==` or `!=`
Compare String text	`a.equals(b)`
Compare whether references share an object	`a == b`
Add to a numeric variable	`+=`
Append to a String variable	`+=` or `+`
Remove or multiply a String with operators	Not supported; use methods
Build text with repeated edits	`StringBuilder`
Use synchronized mutable text methods	`StringBuffer`
Read a whole line of input	`Scanner.nextLine()`